

UniRide CVRPTW

Detaylı Kod Analizi ve Yol Haritasi Raporu

Tarih: 03 Nisan 2026

Analiz Araci: Z.ai

1. Proje Ozeti

UniRide, özel öğrenci taşıma sistemi için geliştirilmiş kapsamlı bir CVRPTW (Kapasiteli Arac Rotalama Problemi Zaman Pencere) çözüm platformudur. Proje, iki ana hedefe hizmet etmektedir: (1) Ticari canlı ürün olarak stabil hibrit algoritmalar ile güvenilir rota planlaması sağlamak, (2) Akademik araştırma ve makale çalışmalarına için SOTA (State-of-the-Art) algoritmalarını test edebilecek bir altyapı sunmak. Bu çift-yollu (Dual-Track) mimari, projenin hem ticari hem de akademik değeri kazanmasını sağlamaktadır.

Proje, Next.js 16 tabanlı frontend ve Python FastAPI tabanlı backend olmak üzere iki ana bileşenden oluşmaktadır. Frontend TypeScript, Tailwind CSS ve shadcn/ui kütüphanesi ile modern bir kullanıcı arayüzü sunarken, backend Python ile çeşitli meta-sezgisel algoritmalar (GA, PSO, GWO, HHO) ve holistik çözümler (OR-Tools, PyVRP, VROOM) kullanarak rota optimizasyonu gerçekleştirmektedir. Supabase/PostgreSQL veritabanı ile veri kalıcılığı sağlanmaktadır.

2. Mimari Yapı

2.1 Genel Sistem Mimarisi

Sistem üç ana katmandan oluşmaktadır: Presentation Layer (Frontend), Business Logic Layer (Backend API) ve Data Layer (Database). Frontend Next.js App Router ile SSR desteği sunmakta ve kullanıcı etkileşimlerini yönetmektedir. Backend Python FastAPI ile optimizasyon algoritmalarını çalıştırmakta ve sonuçları API üzerinden iletmektedir. Veritabanı katmanında Supabase PostgreSQL ile kullanıcı, araç, rota ve ride request verileri saklanmaktadır.

Katman	Teknoloji	Sorumluluk
Frontend	Next.js 16 + TypeScript	Kullanıcı arayüzü, SSR, API proxy
Backend API	Python FastAPI	Optimizasyon algoritmaları, CVRPTW
Database	Supabase PostgreSQL	Veri kalıcılığı, RLS politikaları
Cache	JSON Metadata	Benchmark sonuçları, hash takibi

Tablo 1: Sistem Katmanları ve Teknolojileri

2.2 Algoritma Mimarisi (Çift Pipeline)

Proje, VRP çözümü için iki farklı pipeline yaklaşımı sunmaktadır. Pipeline A (Cluster-First, Route-Second): Öncesi coğrafi kümeleme (Sweep/CW algoritmaları), sonrası her küme için TSP çözümü. Bu yaklaşım, küçük ve orta ölçekli problemler için uygundur. Pipeline B (Route-First, Cluster-Second): Tüm öğrenciler için dev tur (giant tour) oluşturma, sonrası optimal bölme (Split Decoder) ile araç rotalarına ayırma. Bu yaklaşım, büyük ölçekli problemlerde daha verimlidir.

Pipeline	Algoritmalar	Karmaşıklık	Kullanım
Pipeline A	GA, PSO, GWO, HHO	$O(g \cdot p \cdot n^2) + \text{Sweep}$	Orta ölçek
Pipeline B	GA-Split, PSO-Split, GWO-Split, HHO-Split	$O(g \cdot p \cdot n^2) + O(n^2) \text{ Split}$	Tüm ölçekler
Holistik	OR-Tools, PyVRP, VROOM	$O(n^3) / O(n^2 \log n)$	Büyük ölçek

Tablo 2: Pipeline Karşılaştırması

3. Kod Analizi

3.1 Backend Kod Kalitesi

Backend Python kodları yüksek kaliteli ve modüler bir mimariye sahiptir. Strategy Pattern ile algoritmalar birbirinden bağımsız olarak geliştirilebilmekte ve STRATEGY_REGISTRY mekanizması ile dinamik olarak yüklenebilmektedir.

BaseRoutingStrategy soyut sınıfı tüm algoritmalar için ortak bir arayüz tanımlamakta ve kod tekrarını önlemektedir. LinearSplitDecoder sınıfı $O(N \cdot B)$ zaman karmaşıklığı ile dev turleri optimal şekilde bölmekte ve Time-Warp ceza mekanizması ile esnek çözümler üretebilmektedir.

LocalSearch modülü (local_search.py) 2-opt, 3-opt, Or-opt, Swap, Cross Exchange ve Hybrid olmak üzere 6 farklı yerel arama algoritması sunmaktadır. Bu modül, tüm meta-sezgisel algoritmalar tarafından ortak olarak kullanılmakta ve kod tekrarını önlemektedir. Her bir algoritma için akademik kaynak referansları (Croes 1958, Lin 1965, Or 1976, Taillard 1993) dokümanite edilmiştir.

3.2 Frontend Kod Kalitesi

Frontend TypeScript kodları modern React pratikleri ile yazılmıştır. algorithm-constants.ts dosyası tek doğruluk kaynağı (Single Source of Truth) olarak tüm algoritma isimlerini, görünen adlarını ve açıklamalarını içermektedir.

optimizer-service.ts dosyasi Python API ile iletisimi saglamakta ve tip guvenlibir arayuz sunmaktadir. Supabase entegrasyonu ile kimlik dogrulama ve veritabaniiislemleri merkezi olarak yönetilmektedir.

3.3 Akademik Benchmark Altyapisi

Proje, akademik makale hedefi dogrultusunda gelistirilmis bir Smart Benchmark sistemi icermektedir. academic_benchmark klasoru altinda yer alan bu sistem, TSPLIB problemlerini dinamik olarak yuklemekte, algoritma kodlarinin SHA256 hash degerlerini takip etmekte ve degisen kodlar icin otomatik yeniden test baslatmaktadir. run_smart_benchmark.py dosyasi interaktif bir terminal dashboard sunarak kullaniciya test secenekleri sunmaktadir.

4. Tamamlanan Bilesenler

Asagidaki tablo, projede tamamlanmis olan ana bilesenleri ve bunlarin durumlarini ozetlemektedir. Faz 1 kritik duzeltmeler, Faz 1.5 Split algoritmalar, Faz 1.5X Heterojen Filo ve IE Engine, P1-P3 veritabani kal□c□l□□□ ve time window destegi tamamlanmistir. Akademik benchmark altyapisi de son durumda calisir durumdad□r.

Faz	Bilesen	Durum
Faz 1	Kritik Duzeltmeler (API proxy, encoding, auth)	TAMAMLANDI
Faz 1.5	Split Decoder + Pipeline B Algoritmalar	TAMAMLANDI
Faz 1.5X	Heterojen Filo + IE Resource Engine	TAMAMLANDI
P1	Veritabani Kaliciligi (route_plans tablosu)	TAMAMLANDI
P2	Sandbox Backend API	TAMAMLANDI
P3	Time Window Destegi (CVRPTW)	TAMAMLANDI
SOTA	Akademik Benchmark Altyapisi	TAMAMLANDI
Faz 2	Surucu Atama Sistemi	BEKLIYOR
Faz 4	Bildirim Sistemi	PLANLANDI

Tablo 3: Proje Durumu Ozeti

5. SOTA Framework Vizyonu

Projenin SOTA (State-of-the-Art) framework vizyonu, mevcut optimizasyon motorunu 2024-2026 VRP akademik makalelerinde kanıtlanmış yaklaşımlarla donusturmeyi hedeflemektedir. Bu vizyon 4 ana mimari kararı içermektedir: (1) Lineer Zamanlı Split Algoritması - $O(N^2)$ 'den $O(N)$ 'e geçiş, (2) Infeasibility Relaxation - Time-Warp ve kapasite cezaları ile esnek çözümler, (3) ALNS Destekli Sürü Hibridizasyonu - Destroy/Repair operatörleri, (4) Multi-Objective Optimizasyon - Pareto Front çoklu çözümler.

5.1 Faz 1: Core Engine (TAMAMLANDI)

LinearSplitDecoder ile $O(N*B)$ zamanlı bölme algoritması implemente edildi. Time-Warp cezaları ve Soft Capacity mekanizması ile esnek çözüm üretimi sağlandı. Test sonuçlarına göre 150 öğrencilik testte çalışma hızı 1.57 ms'den 0.80 ms'ye düşürüldü (%50 iyileştirme). Motor artık Time-Warp cezalarını tam desteklemektedir.

5.2 Faz 2: ALNS Operatörleri (BEKLIYOR)

Bir sonraki adım, ALNS (Adaptive Large Neighborhood Search) operatörlerinin tasarlanmasıdır. Bu operatörler: Random Removal (çesitlilik), Worst Removal (ceza odaklı), Shaw Related Removal (yakın duraklar), Greedy Insertion ve Regret-2 Insertion (SOTA) şeklindedir. Bu operatörler, sürü algoritmalarının kullanacağı akıllı rota değiştirme ve tamir mekanizmalarını oluşturacaktır.

5.3 Faz 3: Hibritizasyon (BEKLIYOR)

ALNS operatörlerinin HHO, PSO gibi meta-sezgisel algoritmalarla birleştirilmesi planlanmaktadır. Algoritma artık vektör değiştirmek yerine ALNS operatör olasılıklarını (Roulette Wheel) güncelleyecektir. Başarılı operatörler ödüllendirilecek (Adaptive mekanizma). Bu yaklaşım, MO-HHO-ALNS stratejisi olarak literature katkı sağlayabilir.

6. Teknik Borclar ve Eksiklikler

Proje analizi sırasında tespit edilen teknik borclar ve eksiklikler aşağıda listelenmiştir. Bu unsurlar, projenin ilerleyen dönemlerinde ele alınması gereken öncelikli görevleri oluşturmaktadır.

Oncelik	Konu	Aciklama
Orta	Test Coverage Dusuk	Sadece resource_profiler test edildi (20 test). Diger moduller icin test yazilmali.

Orta	Local Search Kısıtlı	Sadece 2-opt varsayılan. 3-opt ve Or-opt daha yaygın kullanılmalı.
Orta	time_matrix Caching Yok	Her istekte DBden yukleniyor. Redis veya bellek ici cache uygulanmalı.
Dusuk	Heterojen Filo Kisim	VehicleConfig semada var ama stratejilerde pasif. Entegrasyon gerekli.
Dusuk	Hybrid Base Strategy Yok	Teknik borc (RI1). Ortak hibrit strateji tabani olusturulmalı.

Tablo 4: Teknik Borclar ve Eksiklikler

7. Yol Haritasi ve Oneriler

7.1 Kisa Vadeli Hedefler (1-2 Hafta)

ALNS operatorlerinin tasarlanmasi ve implementasyonu oncelikli gorev olarak belirlenmistir. optimizer_api/strategies/alns_operators.py dosyasi olusturularak Random Removal, Worst Removal, Shaw Related Removal, Greedy Insertion ve Regret-2 Insertion operatorleri yazilmalidir. Bu operatorler, Strategy Pattern ile moduler bir yapida olmalı ve dev tur (giant tour) üzerinde calismalidir.

7.2 Orta Vadeli Hedefler (3-4 Hafta)

MO-HHO-ALNS hibrit stratejisinin implementasyonu ve mevcut benchmark sistemi ile test edilmesi gerekmektedir. Strateji, HHO algoritmasinin suru zekasini ALNS operator secimi icin kullanmalı ve LinearSplitDecoder ile sonuc uretmelidir. Test sonuclari TSPLIB problemleri üzerinde dogrulanmalı ve akademik makale hazirliklari baslatilmalidir.

7.3 Uzun Vadeli Hedefler (2-3 Ay)

Multi-Objective (Pareto Front) optimizasyon destegi eklenerek Admin kullanicilarina coklu senaryo secenegi sunulmalidir. Bildirim sistemi (Faz 4) implementasyonu tamamlanmalı ve production deployment icin CI/CD pipeline kurulmalidir. Akademik makale yazimi ve uluslararası konferans gonderimi hedeflenmelidir.

8. Sonuc

UniRide projesi, hem ticari bir urun hem de akademik bir arastirma platformu olarak basariyla gelistirilmektedir. Mevcut durumda temel optimizasyon algoritmalari (GA, PSO, GWO, HHO) ve Pipeline B (Split) mimarisi calisir durumdadır. LinearSplitDecoder ile $O(N*B)$ zamanli bolme, Time-Warp cezalari ile esnek cozum uretimi ve Smart Benchmark sistemi ile akademik test altyapisi tamamlanmistir.

Bir sonraki oncelikli gorev, SOTA Framework Faz 2 kapsaminda ALNS operatorlerinin tasarlanmasidir. Bu operatorler, projenin akademik makale hedefine dogru kritik bir adim olusturmaktadir. Teknik borclarin (test coverage, time_matrix caching) kademeli olarak giderilmesi ve surucu atama sisteminin tamamlanmasi, projenin ticari degerini artiracaktir.